

MUSTAFACODE

Lecture 12

Static Data Members, Static Member Functions & Array of Objects
(C++)
Object-Oriented Programming (CS304)

Course Code: **CS304**

Publisher: **mustafacode**

Compiled On: **May 29, 2026**

Lecture 12: Static Data Members, Static Member Functions & Array of Objects (C++)

Chapter 12 — Static Data Members, Static Member Functions & Array of Objects (C++)

◆ 12.1 Static Data Member

Definition

“Ek aisi variable jo class ka hissa hoti hai lekin kisi specific object ka hissa nahi hoti, usay static data member kehte hain.”

Static data members:

- Sab objects ke darmiyan shared hotay hain
- Sirf ek hi copy memory mein banti hai
- Kisi ek object se belong nahi karte

◆ Real Life Example

Socho ek school hai.

Har student ka:

- alag name
- alag roll number

hoga.

Lekin total students count sab ke liye same hoga.

Isliye total students ko static banaya jata hai.

◆ Normal Variable vs Static Variable

✓ Instance Variable

Har object ki apni separate copy hoti hai.

```
class Student{  
    int rollNo;  
};
```

Agar 3 objects bane:

```
Student s1, s2, s3;
```

To memory:

```
s1.rollNo  
s2.rollNo  
s3.rollNo
```

✓ Static Variable

Sirf ek copy hoti hai jo sab share karte hain.

```
class Student{  
    static int noOfStudents;  
};
```

Memory:

```
Class Space:  
noOfStudents
```

Sab objects isi variable ko use karenge.

◆ Syntax of Static Data Member

```
class ClassName{  
  
    static DataType variableName;  
};
```

Example:

```
class Student{  
  
    static int noOfStudents;  
};
```

◆ Defining Static Data Member

Static member:

- class ke andar declare hota hai
- class ke bahar define hota hai

✓ Example

```
class Student{  
  
public:  
    static int noOfStudents;  
};  
  
int Student::noOfStudents;
```

◆ Initializing Static Data Member

Static members ko file scope par initialize kiya jata hai.

✓ Example

```
class Student{  
  
public:  
    static int noOfStudents;  
};  
  
int Student::noOfStudents = 0;
```

◆ Default Initialization

Agar initialization na ki jaye to compiler automatically `0` assign karta hai.

```
int Student::noOfStudents;
```

Equivalent hai:

```
int Student::noOfStudents = 0;
```

◆ Accessing Static Data Member

Static variable ko 2 tareeqon se access kar sakte hain.

✓ 1. Dot Operator `.`

```
Student s1;  
  
s1.noOfStudents = 1;
```

✓ 2. Scope Resolution Operator `::`

```
Student::noOfStudents = 1;
```

◆ Complete Example

```
#include <iostream>
using namespace std;

class Student{

public:

    static int noOfStudents;
};

int Student::noOfStudents = 0;

int main(){

    Student s1;
    Student s2;

    s1.noOfStudents = 5;

    cout << s2.noOfStudents;

    return 0;
}
```

◆ Output

```
5
```

◆ Explanation

`s1` ne value change ki.

Lekin `s2` ne bhi wahi updated value show ki.

Kyuki static variable sab objects ke darmiyan shared hota hai.

◆ Life of Static Data Member

Important Points

- Object ke bina bhi create ho jata hai
 - Program ke end tak memory mein rehta hai
 - Sab objects destroy hone ke baad bhi exist karta hai
-

◆ Example Without Object

```
#include <iostream>
using namespace std;

class Student{

public:

    static int noOfStudents;
};

int Student::noOfStudents = 0;

int main(){

    Student::noOfStudents = 10;

    cout << Student::noOfStudents;

    return 0;
}
```

◆ Output

```
10
```

◆ Explanation

Koi object create nahi hua.

Phir bhi static variable use hua.

◆ Example After Object Destruction

```
#include <iostream>
using namespace std;

class Student{

public:

    static int noOfStudents;
};

int Student::noOfStudents = 0;

int main(){

    {

        Student s1;

        Student::noOfStudents = 5;
    }

    cout << Student::noOfStudents;

    return 0;
}
```

◆ Output

5

◆ Explanation

Object destroy ho gaya.

Lekin static variable memory mein mojood raha.

◆ Practical Use of Static Variable

✓ Counting Total Objects

MUSTAFA CO

◆ Example

```
#include <iostream>
using namespace std;

class Student{

public:

    static int noOfStudents;

    Student(){

        noOfStudents++;

    }

    ~Student(){

        noOfStudents--;

    }

};

int Student::noOfStudents = 0;

int main(){

    cout << Student::noOfStudents << endl;

    Student s1;

    cout << Student::noOfStudents << endl;

    Student s2;

    cout << Student::noOfStudents << endl;

    return 0;

}
```

◆ Output

```
0
1
2
```

◆ Step By Step Explanation

◆ Program Start

```
noOfStudents = 0
```

◆ s1 Create Hua

Constructor call hua:

```
noOfStudents++;
```

Value:

```
1
```

◆ s2 Create Hua

Phir constructor call hua.

Value:

```
2
```

◆ Problem in Above Design

noOfStudents

public hai.

Koi bhi bahar se change kar sakta hai.

Ye achi design nahi.

MUSTAFA CO

◆ Better Design Using Private Static Variable

```
#include <iostream>
using namespace std;

class Student{

private:

    static int noOfStudents;

public:

    Student(){

        noOfStudents++;

    }

    static int getTotalStudents(){

        return noOfStudents;

    }

};

int Student::noOfStudents = 0;

int main(){

    Student s1;
    Student s2;

    cout << Student::getTotalStudents();

    return 0;

}
```

◆ Output

2



12.2 Static Member Function



Definition

“Wo function jo class ke members ko access kare lekin kisi specific object ke through call na ho, usay static member function kehte hain.”

◆ Important Points

- Static functions static members ko access karte hain
 - Object ke bina call ho sakte hain
 - `this pointer` nahi hota
 - Non-static members access nahi kar sakte
-

◆ Syntax

```
static returnType functionName();
```

◆ Example

```
#include <iostream>
using namespace std;

class Student{

private:

    static int totalStudents;

public:

    Student(){

        totalStudents++;
    }

    static int getTotalStudents(){

        return totalStudents;
    }
};

int Student::totalStudents = 0;

int main(){

    Student s1;
    Student s2;

    cout << Student::getTotalStudents();

    return 0;
}
```

◆ Output

2

◆ Why Static Function Cannot Access Non Static Variable

✗ Wrong Example

```
class Student{  
  
    int rollNo;  
  
public:  
  
    static int getRollNo(){  
  
        return rollNo;  
    }  
};
```

◆ Error Reason

Static function ke paas object nahi hota.

Isliye compiler nahi janta kis object ka `rollNo` access karna hai.

12.3 this Pointer and Static Functions

◆ this Pointer

`this` ek hidden pointer hota hai jo current object ka address store karta hai.

◆ Important Point

✓ Normal Function

```
void show(){  
}
```

Compiler hidden way mein:

```
show(this);
```

pass karta hai.

✗ Static Function

Static function ko `this pointer` nahi milta.

Kyunki static function kisi object se belong nahi karta.

12.4 Global Variable vs Static Member

◆ Global Variable

```
int totalStudents;
```

Program ke har hissa se accessible hoti hai.

◆ Problem

- Data unsafe ho jata hai
- Information hiding break hoti hai
- Koi bhi value change kar sakta hai

◆ Better Approach

Static private member use karo.

12.5 Array of Objects

◆ Important Rule

Agar objects ka array banana ho to class mein default constructor hona zaroori hai.

◆ Example 1

```
class Test{  
  
};  
  
int main(){  
  
    Test array[2];  
  
    return 0;  
}
```

✓ OK

Compiler khud default constructor bana deta hai.

◆ Example 2

```
class Test{  
  
public:  
  
    Test(){  
  
    }  
};  
  
int main(){  
  
    Test array[2];  
  
    return 0;  
}
```

✓ OK

Default constructor mojud hai.

◆ Example 3

```
class Test{  
  
public:  
  
    Test(int i){  
  
    }  
};  
  
int main(){  
  
    Test array[2];  
  
    return 0;  
}
```

✗ Error

◆ Reason

Compiler ko har object ke liye value chahiye.

Lekin array banate waqt koi value provide nahi hui.

◆ Example 4

```
class Test{  
  
public:  
  
    Test(int i){  
  
    }  
};  
  
int main(){  
  
    Test array[2] = {Test(1), Test(2)};  
  
    return 0;  
}
```

✓ OK

Kyuki dono objects explicitly initialize hue.

◆ Example 5

```
class Test{  
  
public:  
  
    Test(int i){  
  
    }  
};  
  
int main(){  
  
    Test a(1), b(2);  
  
    Test array[2] = {a, b};  
  
    return 0;  
}
```

✓ OK

Pehle initialized objects array mein copy hue.



Summary

Concept	Important Point
Static Variable	Sirf ek copy hoti hai
Static Member	Sab objects share karte hain
Static Function	Object ke bina call hota hai
this Pointer	Static function mein nahi hota
Array of Objects	Default constructor zaroori hota hai